Distributed Database Algorithms using RAFT to enhance Quorum-Based Log Replication

Albert Lin
Department of Computer Science
San Jose State University
San Jose, California
albert.lin01@sjsu.edu

Benjamin Ha
Department of Computer Science
San Jose State University
San Jose, California
benjamin.k.ha@sjsu.edu

Vedant Sawal
Department of Computer Science
San Jose State University
San Jose, California
vedant.sawal@sjsu.edu

***Abstract*—This paper proposes the implementation of a distributed database, focusing on the main challenges in distributed computing, such as fault tolerance, consistency, and concurrency control. The system will be replicated on a MySQL database, implemented with distributed mechanisms like quorum based log distribution and RAFT learned from recent papers to ensure data integrity and availability in the event of node failures.**

Keywords—distributed; computing; systems; databases; consistency; fault tolerance; concurrency control

I. Introduction

Traditional databases, or centralized databases, store data on a single server or cluster managed by a single entity. While this architecture simplifies management, it suffers from significant scalability issues, including bottlenecks in server and network performance, security risks, and a single point of failure. Vertical scaling, which involves adding more resources to a single node, is often cost-prohibitive compared to distributed systems that can provide more scalable, cost-effective solutions by utilizing smaller, modular nodes. This research is motivated by two key objectives: (1) investigating whether decentralizing traditional databases using distributed systems concepts can provide measurable benefits in scalability, efficiency, and resilience, and (2) overcoming the technical challenges of implementing such a decentralized system. Specifically, we aim to explore whether these concepts can be applied to improve load balancing, reduce bottlenecks, and ensure system reliability in distributed database architectures. Decentralizing data across multiple nodes addresses traditional bottlenecks but introduces new challenges. Key among these is the complexity of load balancing, especially in systems that use hashing for data distribution. While hashing can efficiently balance load, it can also lead to fragmentation and computational overhead in cross-shard operations. Additionally, although hashing minimizes the risk of data collisions, the rare occurrence of a collision can have serious implications for data integrity. Balancing the advantages of decentralization with these challenges is central to developing a viable and efficient solution.

II. Literature Review

In the quest to understand the challenges in distributed database systems, we reviewed several key papers that have made notable advancements in the field. Below are brief summaries of some of the most influential works:

A. FlexiRaft: Flexible Quorums with Raft

The Raft consensus algorithm traditionally relies on a static list of pre-selected nodes to form a quorum for leader election. However, in the work by *Meta (Facebook)*, the authors introduce **FlexiRaft** [1], which enhances Raft by allowing for dynamic quorum membership. Unlike the static quorums in traditional Raft, **FlexiRaft** enables the membership of the voting system to be configurable at runtime. This innovation introduces two modes:

- **Static**: Default to Raft's system of preselected nodes.
- **Dynamic**: The nodes involved in the voting system are changeable at runtime, enabling flexibility and reducing the dependency on the entire set of nodes.

B. RaftOptima: An Optimized Raft with Enhanced Fault Tolerance and Increased Scalability

RaftOptima [2] augments the Raft algorithm with a novel proxy leader concept. Instead of the elected leader directly broadcasting AppendEntries to all followers, **RaftOptima** delegates this responsibility to **proxy leaders**. This reduces the leader's communication load, increasing system scalability and robustness. Key contributions include:

- **Scalability**: Reduces the leader's direct communication fan-out by using proxy leaders.
- **Latency**: Demonstrates up to 60% lower latency in simulations with clusters up to 25 servers.

- **Robustness**: Improves system performance under partial network partitions.
- **Correctness**: Verified using TLA+ formal methods alongside simulation results.

C. QuORAM: A Quorum-Replicated Fault-Tolerant ORAM Datastore

The **QuORAM** protocol [3] addresses a critical gap in existing **Oblivious RAM (ORAM)** datastores, which are typically not fault-tolerant. **QuORAM** introduces a quorum-based replication protocol that provides:

- **Fault tolerance**: Through decentralized, lock-free replication, QuORAM ensures data integrity even when trusted proxies or storage servers fail.
- **Security**: It obfuscates access patterns, preventing attacks that rely on monitoring those patterns.

D. Performance Enhancement of Distributed Processing Systems Using Novel Hybrid SHARD Selection Algorithm

In the paper by *Zhang et al.* [4], the authors propose a **Hybrid Shard Selection Algorithm (HSSA)** to improve the performance of distributed databases at scale. The HSSA focuses on minimizing query latency and optimizing shard selection through dynamic adjustments. Key features of this approach include:

- **Hybrid sharding**: Combines static sharding with dynamic adjustments based on query-specific relevance.
- **Smart catalog**: Maintains a **Central Sample Index** to dynamically track high-traffic data, improving access efficiency.
- **Retrieval-Driven Dynamic Shard Estimation (ReDDE)**: Predicts query relevance and load balancing by learning from historical performance data.

E. Multi-Attribute-Based Self-Stabilizing Algorithm for Leader Election in Distributed Systems

The **Multi-Attribute Self-Stabilizing Leader Election (MA-SSLE)** algorithm, introduced by *Tan et al.* [5], improves leader election in distributed systems by considering multiple attributes when electing a leader. The key features include:

- **Self-stabilization**: The algorithm ensures that the system converges to a valid state and remains stable even if faults occur.
- **Partition tolerance**: In the event of network partitions, **MA-SSLE** elects local leaders, and the system re-stabilizes once connectivity is restored.

III. Project Design

Our distributed database system is designed to address core distributed-systems challenges through a combination of FlexiRaft-based replication and HSSA data sharding. In this section, we discuss the design of our distributed database system, including the system's design goals, key components and algorithms, and the overall architecture with its interacting modules.

A. Design Goals

1) Heterogeneity

The current implementation addresses algorithmic heterogeneity primarily at the data model level through shard partitioning by original language. Each shard forms its own independent consensus group, simulating the federated handling of diverse data types, which is a foundational step toward full heterogeneity support. The system's modular design, including configurable replica sets and pluggable adapters for future database backends, provides architectural readiness for heterogeneous deployment scenarios.

Specifically:

(a) Shard-level partitioning by language embodies conceptual data heterogeneity, with flexible configuration of replica memberships.

(b) The abstraction of consensus and replication mechanisms (FlexiRaft logic, per-shard orchestration) decouples system operations from underlying data semantics, permitting future adaptation to mixed storage backends or process types.

(c) Modular code and test structure create avenues for experimenting with diverse cluster topologies and data domains.

Current support for heterogeneity is limited to in-process simulation of language-domain separation and modularity at the software layer. The repository does not yet implement cross-platform, cross-language, or multi-engine cluster components, nor does it offer runtime adaptations for hardware or protocol diversity. As such, this work demonstrates architectural and algorithmic preparation for heterogeneity, but does not claim proven interoperability or runtime support for heterogeneous environments.

2) Openness

The current implementation supports architectural openness by enabling reconfigurable consensus and data-placement layers, coupled with explicit primitives for guided, safe membership transitions. Specifically:

(a) Flexible quorum election logic (see wrapper/flexiraft.py) allows cluster membership and voting rules to be adapted without monolithic, static configurations.

(b) Historical vote inference (see wrapper/flexiraft.py and tests/vote_inference_test.py) provides mechanisms for reasoning about safe membership changes over time, supporting transitions with data consistency assurances.

(c) Explicit sharding (see wrapper/sharding.py) decouples data movement from membership changes, enabling coordinated and transparent transitions.

Current openness assurances focus on algorithmic and architectural support verified via unit tests and single-process simulation. The repository does not yet implement dynamic cluster join/leave orchestration, real-time membership reconfiguration protocols, or distributed open-access APIs. Thus, while the code demonstrates readiness for openness in core logic, it does not yet provide a fully dynamic or permissionless operational environment.

3) Security

Current security assurances in our implementation are limited to aspects of algorithmic correctness and resilience in the face of accidental faults and process crashes. The system employs strict majority consensus for leader election and data updates via the FlexiRaft protocol (wrapper/flexiraft.py), with thorough validation through unit and integration testing of election behavior and crash recovery scenarios (tests/test_flexiraft.py, test_crash_recovery.py). These measures ensure that, despite node or leader failures, the system remains consistent and avoids undefined or partially applied states. However, the system does not implement mechanisms for authenticated transport, message signing, encrypted data storage, persistent state integrity, membership authentication, or cryptographic key management. There are no adversarial tests, audit trails, or controls for unauthorized access. Consequently, real-world security guarantees—such as resistance to network attacks, unauthorized data access, or malicious node participation—are not claimed in this work. Present evidence of security is confined to internal correctness and survivability under controlled failure conditions within a trusted simulation environment.

4) Failure Handling

The project's failure handling mechanisms leverage per-shard partitioning and consensus protocols to provide robust availability in the face of node and group failures. Failure isolation is achieved via explicit sharding: when a shard (e.g., corresponding to a particular language group) experiences node outages or leadership loss, only that shard is affected, and the operation of other shards continues unimpeded. This approach ensures that client queries routed to healthy and relevant shards remain uninterrupted, enhancing system availability and resilience.

Leader failure and recovery within shards are managed by the FlexiRaft consensus algorithm, which enables rapid leader re-election and supports both static and dynamic quorum specifications. While quorum adjustment requires pre-defined configuration, FlexiRaft's parallel leader election and failover logic ensures continued operation within healthy partitions.

Specifically:

(a) Partitioned failure domains isolate faults to individual shards, preventing cascading effects.

(b) FlexiRaft-based consensus provides leader re-election and fault-tolerant data replication within shards.

(c) Deterministic unit and system tests validate that, upon shard failure, unaffected shards remain available for query and data operations.

Present assurances are limited to architectural and simulated test-time resilience. The codebase does not presently implement automatic runtime reconfiguration, dynamic quorum resizing, or network-distributed failover; these remain areas for future development.

5) Concurrency

Our implementation addresses concurrency by making shard-level command sequencing and election decision logic explicit and testable, and by providing concrete primitives and tests that demonstrate correct reasoning about interleaved election inputs without assuming a single global execution order.

Concretely, the project supplies

(a) per-shard leader election using FlexiRaft quorum logic that tolerates partial and overlapping responses

(b) a historical-vote inference helper that helps interpret concurrent term histories and guide safe leader selection

(c) explicit sharding so independent shards form separate concurrency domains, allowing concurrent client activity across languages.

In addition, FlexiRaft enables concurrency through separate and customizable quorums for elections and data commits: the election quorum and commit quorum can be tuned per shard (subject to quorum-intersection safety), allowing leader elections and log replication/commit acknowledgments to proceed with minimal head-of-line blocking. This decoupling lets shards continue committing while other shards are electing leaders, and allows operators to balance latency, throughput, and fault tolerance per shard and workload.

6) Quality of Service

The implementation improves Quality of Service primarily through per-shard isolation, flexible-quorum leader election, and rigorous crash recovery mechanisms. Availability and fault tolerance are explicit and testable properties, not inferred from implicit behavior. Specifically:

(a) Flexible-quorum election logic (wrapper/flexiraft.py) enables leader failover and configurable availability tradeoffs.

(b) Explicit sharding by original language (main.py), coordinated per-shard membership (db/flexiraft_coordinator.py), ensures that requests for independent shards may be served in parallel.

(c) Extensive unit and integration testing (tests/test_flexiraft.py, tests/test_flexiraft_integration.py), along with system-level crash and failover tests (test_crash_recovery.py), validate availability invariants under simulated conditions and partial failure scenarios.

The current assurances are restricted to the correctness of the in-process simulation and logical invariants verified by these tests. The system, in its present form, does not implement runtime QoS guarantees, measured latency or throughput benchmarks, service-level objectives, or mechanisms for request prioritization, rate limiting, or backpressure. Future work will address these gaps.

7) Scalability
The current implementation supports architectural scalability by explicitly separating data placement and consensus logic and by providing primitives that allow horizontal expansion at the shard and replica group levels. Specifically:

(a) Explicit sharding by original language establishes independent scaling domains.

(b) Flexible quorum logic (wrapper/flexiraft.py) supports configurable availability and replication tradeoffs across replica groups.

(c) Modular database adapters and in-process node state management (see db/node_*) facilitate manual instantiation of additional replica processes for scale experiments.

Current scalability assurances are limited to architectural and algorithmic readiness demonstrated via single-process simulation and testing. The repository does not yet include a distributed RPC layer, dynamic cluster orchestration, rebalancing or autoscaling mechanisms, or empirical scaling results. Accordingly, this work does not claim to demonstrate proven scalability or elastic operation at production scale.

8) Transparency

The implementation maintains logical transparency for clients by exposing a unified query interface, abstracting the underlying routing, replication, and failover mechanisms. Algorithmic behavior, test scenarios, and per-node state are made explicit and reproducible, facilitating tractable reasoning for reviewers and operators.

Specifically:
(a) The project provides readable, paper-aligned election logic and a historical-vote inference helper (wrapper/flexiraft.py, wrapper/flexiraft_helper.py), exercised via deterministic unit and integration tests.

(b) Explicit per-node directories and example state (db/node_*, state.json) expose persisted state for inspection and reproduce test scenarios for analysis.

(c) Modular code and comprehensive test coverage enable reproducible simulation of election and replication workflows.

Transparency is presently assured at design-time and test-time: the codebase is inspectable, and the tests are deterministic and reproducible. The current system does not provide runtime observability features such as structured logging, metrics, distributed tracing, or a cluster-wide audit trail; nor does it produce standardized configuration snapshots or human-readable audit exports. As such, the project does not yet claim operational transparency at production scale.

B. System Components and Algorithms

1) Sharding and Data Partitioning
Our goal was to segment the global dataset into physical nodes (“shards”) for modular, scalable access and improved resilience.

Key Operations:

- Shard Routing: Selects and assigns the target shard for a given query (write_entry() in main.py).
- Leadership Assignment: Determines and retrieves the current leader node for a shard (get_leader() in db/flexiraft_coordinator.py).
- Consensus Quorum Initiation: Starts the quorum voting process among shard replicas (raft_quorum() in db/flexiraft_coordinator.py, flexiraft_leader_election() in wrapper/flexiraft.py).
- Log Replication: Persists write operations in distributed log files for each replica (append_raft_log() in main.py).
- Commit Updates: Finalizes data changes across replicas when quorum is achieved (update_commit_index() in main.py).

We guarantee that sharding enables independent failure domains, parallel write throughput, and lays the groundwork for scalable distributed data placement.

2) FlexiRaft Consensus and Leader Election
Our goal was to enable robust leader election and fault-tolerant consensus through flexible, per-shard quorum specifications.

Key Operations:

- Election Initiation: Begins leader elections for a given shard, either on schedule or in response to failure (elect_leader_for_group() in db/election_manager.py).

- Vote Request and Collection: Simulates and records vote requests and responses for quorum establishment (simulate_vote_request() and collect_votes() in db/flexiraft_coordinator.py).

- Quorum Evaluation: Determines leader election status, applying Raft and FlexiRaft logic to support both static and dynamic quorums (flexiraft_leader_election() in wrapper/flexiraft.py).

- Leader Commitment: Updates all shard replicas with the new leader after successful election (update_all_replicas_with_leader() in db/flexiraft_coordinator.py).

- Failover and Retry Logic: Retries elections with backoff if consensus is not reached, supporting robustness in failure cases (elect_leader_for_group() in db/election_manager.py).

We guaranteed that consensus and leader election procedures ensure system safety and liveness, with configurable availability and resilience parameters.

3) FlexiRaft Historical Vote Inference
Our goal was to evaluate prior voting history to identify safe leadership candidates without arbitrary assignment.

Key Operations:

- Candidate Group Mapping: Constructs tables for candidate groups and quorum membership (_candidate_group_map() in wrapper/flexiraft_helper.py).

- Current Group Counts Calculation: Computes granted votes, received responses, and group size for each potential leader group (current_group_counts() in wrapper/flexiraft_helper.py).

- Term/Group Historical Mapping: Establishes mapping of group votes to terms for evaluating historical majority (term_group_candidates() in wrapper/flexiraft_helper.py).

- Potential Leader Selection: Iteratively evaluates candidates and assigns leadership only to those demonstrating majority support (GetPotentialNextLeaders() in wrapper/flexiraft_helper.py).

- Termination Conditions: Distinguishes between detected candidate majorities (POTENTIAL_NEXT_LEADERS_DETECTED) and the absence of sufficient support (ALL_INTERMEDIATE_TERMS_DEFUNCT).

We ensured leadership safety by requiring historical group majority, preventing arbitrary or unsafe leader selection.

4) Safety and Durability

Our goal was to guarantee data correctness and durability under failure scenarios.

Key Operations:

- Majority-Based State Changes: Only commit log updates and leadership transitions when strict quorum and safety conditions are met (flexiraft_leader_election() and update_commit_index()).

- Persistent Logging: Write operations and election events are logged so that the system can recover state reliably after failure (append_raft_log() in main.py, state files in db/node_*/state.json).

We guaranteed that all state transitions and data updates require proven majority support, preserving safety, reliability, and durability.

IV. System Architecture

The distributed database system follows a multi-layered architecture consisting of consensus, replication, routing, and coordination layers. In our example, we designed our system to storage a library of novels in 32 individual SQLite database instances organized into 8 language-based shard groups (Chinese, Japanese, Korean, Malay, Filipino, Indonesian, Khmer, and Thai), with each shard replicated 4 times across different physical nodes, simulated on-premise as file directories.

A. High-Level Design and Topology

The system employs a ring-based replica placement strategy where each node hosts exactly 4 shards, ensuring balanced load distribution. For example, node_0 hosts shards for Chinese (zh), Thai (th), Japanese (ja), and Khmer (km) languages. This arrangement guarantees that any two consecutive nodes in the configuration share exactly two common shards, optimizing for network locality while maintaining geographical diversity.

The replica placement follows a carefully designed topology defined within *db/node_config.json*. Each language shard maintains a four-node replica set with a primary replica ordering. For example, the Chinese shard (zh) is replicated on node_0, node_7, node_1, and node_2; where node_0 serves as the preferred primary location. This deterministic ordering aids in leader election efficiency and failure recovery predictability.

B. FlexiRaft Consensus Algorithm Implementation

The system implements FlexiRaft, an advanced consensus algorithm that extends traditional Raft with dynamic quorum capabilities by disconnecting election quorum requirements from write quorum requirements. This enables the system to maintain availability under different failure scenarios.

The algorithm defines two distinct quorum types through a modified **majority_count()** function:

1) Election Quorum: Requires a true majority (3 out of 4 replicas) for leader election, ensuring strong consistency for execution decisions.
2) Write Quorum: Requires only a pre-determined number (2 out of 4 replicas) for write operations, enabling for reduced latency and stretching fault tolerance buffer zone to up to 2 replicas failures in certain circumstances.

This split-quorum approach represents a significant departure from traditional Raft implementations, which typically require the same quorum for both elections and writes. The trade-off allows the system to maintain availability for write even when a minority of replicas are unreachable, while remaining strict requirements for leadership to prevent split-brain scenarios.

C. FlexiRaft Algorithm Components

The FlexiRaft implementation consists of two primary algorithms working in tandem:

1) Leader Election (*wrapper/flexiraft.py*): This algorithm manages the election process through multiple phases:
    a) Static quorum evaluation for configured quorum specifications
    b) Dynamic quorum evaluation using pessimistic quorum status
    c) Last-known-leader optimization for immediate term successors
    d) Iterative leader inference through historical vote analysis

The algorithm maintains a Replica Set Topology structure that describes the groups and their member voters, enabling flexible quorum definitions across different group configurations.

2) Potential Next Leader (wrapper/flexiraft_helper): This supporting algorithm implements historical vote analysis to infer potential leaders from previous terms. It examines voting history records maintained by each node to determine if intermediate leaders existed between the last known leader and the current term. The algorithm returns one of the three statuses:
    a) ALL_INTERMEDIATE_TERMS_DEFUNC: No valid leaders existed in intermediate terms
    b) WAITING_FOR_MORE_VOTES: Insufficient information to make a determination
    c) POTENTIAL_NEXT_LEADERS_DETECTED: Identified candidate leaders from historical roles

D. Vote Collection and Response Handling

The FlexiRaft coordinator (*db/flexiraft_coordinator.py*) bridges the consensus algorithm with the distributed infrastructure. It simulates vote requests by reading replica state files and collecting quorum response objects containing:

- Voter identification: node_id and group
- Current term information
- Vote granted/denied status
- Complete voting history for Potential Next Leaders processing

Vote granting follows strict Raft principles: A replica grants its vote if the candidate's term exceeds the replica's current term, or if the terms match and the replica hasn't been voted or already voted for the same candidate. This ensures the critical Raft property that each replica votes at most once per term.

E. State Management Design

Each replica maintains a comprehensive state machine persisted in *state.json files* (*db/state_manager.py)*.

The state includes:

- Identity: node_id and shard assignment
- Raft State: Current term, voted_for candidate, role (leader/follower/candidate)
- Log Indices: commit_index (highest committed index) and last_applied (highest index applied to database)
- Leadership: current_leader identifier and last_heartbeat timestamp
- History: Complete voting_history for Potential Next Leaders processing
- Configuration: Election timeout parameters

The state manager implements both synchronous and asynchronous persistence modes. Leaders use synchronous writes with file locking via fcntl.flock() to ensure durability before acknowledging operations. Followers use asynchronous writes via daemon threads to avoid blocking the consensus protocol, with error handling for race conditions during rapid elections and system shutdown.

F. Raft Log Management

The Raft logs provides atomic append operations using file locking and supports bulk appends for efficiency. Log reconciliation implements the standard Raft approach: finding the divergence point between leader and follower logs, truncating the follower's log after divergence, and copying the leader's authoritative entries.

The system also supports ghost entries, where uncommitted log entries that may exist on replicas that failed to achieve quorum. Ghost entries can be overwritten during log reconciliation without compromising safety, as they were never committed.

G. Write Execution Flow

Write operations are where the complexity of this design lies. It follows a multi-phase protocol orchestrated by the main coordinator in this order:

1) Leader Discovery: System ensures a valid leader exists for the target shard through on-demain election. If no leader is recorded or the leader is unavailable, an election is triggered before proceeding.
2) Log Replication: The operation is prepared as a Raft log entry containing the SQL operation and parameters. The coordinator then executes parallel writes to all 4 replicas, attempting to append the entry to each replica's raft log simultaneously.
3) Quorum Verification: As replicas respond, the coordinator counts successful appends; if at least 2 replicas successfully append the entry, the transaction commits. The coordinator updates the commit_index in the state files of all successful replicas.
4) Retry Logic: If quorum fails, the system checks for shard death (<2 replicas available) before retrying. An exponential backoff with up to 5 retry attempts allow transient failure to resolve.

H. Asynchronous Updates

By separating the log replication from database application, the write protocol only needs to append to raft logs and update commit indices; it does not directly modify SQLite databases. Instead, separate commit daemons will eventually apply committed entries asynchronously.

1) Latency Reduction: Write operations return after write quorum, skipping database I/O process
2) Failure Isolation: Database corruption or slow database operations doesn't block the consensus protocol
3) Batch Optimization: Daemons can batch multiple log entries into single database transactions

I. Background Daemon Architecture

1) Commit Daemon runs as a background thread for each node, managing all shards hosted on that node. Each daemon operates in a continuous loop of 2 second intervals, checking each shard for committed but unapplied entries. For each shard, the daemon
    a) Loads current state to read commit and last applied indices
    b) If shard is leader, updates heartbeat timestamp
    c) Execute SQL statements where last_applied < index ≤ commit_index
    d) Update last_applied after successful write
2) The Recovery Daemon implements automatic failure detection and recovery by performing checks every 30 seconds:
    a) Leader Health Monitoring: The daemon checks the leader's last heartbeat timestamp. If the leader hasn't updated its heartbeat within the timeout window, the daemon triggers a reelection for that shard group.
    b) Replica Lag Detection: The daemon compares each follower's commit_index against the leader's commit_index. If the follower's index lags significantly, the daemon initiates recovery to allow for data catch-up.
    c) Pull-Based Recovery: Data catch-up triggers a pull-based log replication where the recovery daemon copies missing log entries from the leader's raft log to the follower's log, then updates the follower's commit_index for writing into the database eventually, bringing it back up to speed.
    d) Recovered Node Detection: After a node recovers from a crash, it will reenter the replica ring, but its commit_index will be

significantly higher than its last_applied index. A custom threshold is set so that when the commit_index > last_applied + threshold, a proactive recovery will be initiated without waiting for the replica to become significantly behind

J. Election Management

The election manager orchestrates leader elections, supporting both upfront election of all leaders and on-demand elections triggered by write operations.

For system initialization, the election manager can elect leaders for all 8 shard groups in parallel, and partial elections can also be held when the recovery daemon detects a loss of leader in a specific shard group.

K. Sharding

The system employs content-based sharding using the original_language as the key as a routing algorithm. This approach provides several advantages:

1) Query Locality: Queries only access one shard group, usually the leader shard
2) Balanced Distribution: Uniformed distribution of shards help optimize traffic
3) Horizontal Scalability: System can be sharded with new shard groups, further dissecting access points.

The Hybrid Shard Selection Algorithm further detects hotspots of most accessed shards by introducing a dynamic layer.

Our database logic deliberately avoids foreign keys and complex constraints to minimize cross-shard dependencies and reduce database locking contention. Auto-incrementing identifiers have to be local to each shard, with uniqueness guaranteed within a language instead of globally.

V. Evaluation

A. Correctness & Complexity

1) FlexiRaft Main Algorithm

- Intuitive Description

  The main algorithm for FlexiRaft leader election decides who becomes leader in the current term using the smallest quorum that is still safe. The inputs are the vote responses from replicas with each voter labeled by its group, the cluster topology grouped by locality or failure domains, the current term, and an optional last known leader from the previous term. In static mode the algorithm simply checks the configured quorum formula and returns won if any option is satisfied, undecided if an option is still achievable, and lost otherwise. In dynamic mode the algorithm tries three checks in order. It first tests a conservative condition that requires a majority in every group. If that is not already achieved it tries the last known leader fast path in which a candidate wins in the next term after the last leader if it has a majority inside the last known leader group. If neither condition is enough the algorithm calls GetNextPotentialLeaders() which analyzes voter histories and returns the groups that legitimately constrain possible successors. The main algorithm then requires majorities only in those detected groups. This approach is good because it reduces latency in the common case through the last known leader fast path, adapts to irregular vote patterns through GetNextPotentialLeaders(), and never declares a leader unless the evidence proves a safe quorum.

- Discussion of Correctness

  Safety requires that at most one leader be chosen in a term. In each group a voter grants at most one vote per term, therefore two distinct candidates cannot both hold a majority in the same group in the same term. If two candidates both claimed victory by the conservative rule that demands a majority in every group then in any particular group both would have to hold a majority and that would require some voter to grant two votes in the same term which is impossible. If two candidates both claimed the last known leader fast path in the same term then both would need a majority inside the same last known leader group which would again force a double vote which cannot happen. When GetNextPotentialLeaders() is used it returns specific groups that must certify a legitimate successor based on the voting histories. The main algorithm then requires majorities in those returned groups. Two candidates cannot both satisfy disjoint terminal requirements in the same term because the construction of the returned sets forces an intersection with any rival certification path. Hence no two distinct candidates can both be returned as winners in the same term. Because election quorums intersect with the commit quorums from the prior term, any new leader necessarily contains all entries that could have been committed, so state machine safety is preserved.

  Liveness requires that some candidate eventually becomes leader once the system is stable. With randomized timeouts one election starts while others wait. If the candidate has enough responses for the conservative rule or for the last known leader fast

path it wins immediately. Otherwise additional votes and the guidance from GetNextPotentialLeaders() eventually identify the necessary groups and the candidate receives the required majorities in those groups. Under bounded message delay and with at least one group that can form a majority, some rounds will complete and the algorithm returns won. Thus a leader is eventually elected.

- Analysis of Complexity

  Let there be G groups and a total of N voters in the replica set and let d be the round trip time on the network inside the set. The time to decide is dominated by one or two message rounds after the needed votes arrive. The conservative rule decides once the candidate has majorities in all groups which is typically slower. The last known leader fast path decides once a majority arrives from the single last known leader group and is usually the fastest. When GetNextPotentialLeaders() is involved, decisions still occur after the required groups produce their votes and the number of iterations is bounded by G because no more than all groups can be detected. In all cases the latency after votes arrive is on the order of d. The number of messages in one election round is on the order of N since each candidate sends a vote request to every voter and receives up to N replies. The main algorithm stores the current set of responses and tallies per group, which costs on the order of N for responses and on the order of G for aggregated counts, while voting histories are bounded by the number of responses in the round. In your deployment with four replicas per set the last known leader fast path needs three votes from that single group and therefore often completes after the first wave of replies, while the conservative rule would need three votes from every group and is rarely the winning path under load.

Operation	Complexity
Vote tally aggregation	O(N)
Pessimistic quorum check	O(G)
LKL fast-path check	O(1)
Space	O(NK)

2) FlexiRaft GetNextPotentialLeaders()

The FlexiRaft GetNextPotentialLeaders() function provides a mechanism to identify which nodes in a distributed system are best suited to become the next leaders. Rather than simply considering all current members or arbitrary candidates, this function leverages historical information about how quorums have voted in recent leader elections and log commit operations. By analyzing patterns in past quorum participation and node voting behavior, the algorithm determines which nodes have consistently contributed to successful consensus and are therefore most likely to maintain safety and liveness if promoted to leader. This approach increases the likelihood that a newly elected leader will rapidly form quorums and continue progress, minimizing disruption and enhancing overall system reliability.

- Discussion of Correctness

FlexiRaft's GetNextPotentialLeaders() ensures safety by only selecting nodes that have participated in recent successful quorums, which implies these candidates possess the most up-to-date log entries and can immediately form quorums with intersection properties. This reduces the risk of electing a leader with stale or divergent state, thereby preventing split-brain scenarios or conflicting log commits. As a result, the system maintains the Raft guarantee that committed entries remain durable and consistent.

The function also ensures liveness by prioritizing nodes that have actively participated in recent quorum decisions, making it more likely that the next leader can quickly assemble a functional quorum for processing client requests and committing new log entries. By leveraging historical voting data, FlexiRaft avoids repeatedly electing leaders who are unable to gather sufficient votes, minimizing leader election stalls and facilitating continued system progress.

- Analysis of Complexity

Let N denote the number of nodes in the cluster, and K the number of recent quorums analyzed based on historical voting data. The function getNextPotentialLeaders() examines each node's participation within these quorums to determine leader candidacy.

(a) Time Complexity: In a typical implementation, the selection process traverses all nodes and, for each node, inspects its presence within each of the K quorums. This results in a time complexity of O(NK). If the algorithm requires validating intersection properties between quorums for each candidate, the worst-case complexity may increase to $O(NK^2N)$. Such exhaustive checks are uncommon in practice and can often be optimized using bitwise operations or efficient data structures. With optimized set operations (such as bitsets), intersection checks can be performed in O(1) or O(N / w) time, where w is the machine word size.

(b) Space Complexity: The space required for storing the participation matrix or historical quorum data is O(NK). The output candidate set occupies up to O(N) additional space.

(c) Complexity Summary:

Operation	Complexity
Candidate Selection	O(NK)
Quorum Selection	$O(NK^2N)$ (worse case)
Space	O(NK)

3) Data Sharding

In our distributed database system, data sharding divides the logical database into eight shard groups, each of which is further replicated (four replicas per group) for durability and fault tolerance, resulting in 32 shards total distributed amongst simulated nodes. Each shard group is assigned a dynamically elected leader using a Raft-based consensus protocol. When an application issues a request (such as a write or query), the sharding layer performs key-based routing to determine the target shard group. The request is then forwarded to the leader node of the appropriate group. Write operations are coordinated using quorum-based replication (FlexiRaft), with log shipping and commit index tracking to ensure all replicas for a shard remain consistent. This approach enables scalable modular access patterns and robust geographic and fault-tolerance characteristics.

- Discussion of Correctness

The implementation ensures safety by using FlexiRaft to coordinate every write within a shard group. Before any update is committed, it must be written to the Raft log and acknowledged by a quorum of replicas in the group. Consistency is guaranteed as only the shard group's elected leader initiates log appends and commit operations, preventing conflicting writes and avoiding split-brain scenarios. The sharding router always directs each write to the correct group and ensures no cross-shard contamination occurs.

Liveness is maintained as the router always directs reads and writes to the active leader of each shard group. If a leader fails, the Raft algorithm handles leader re-election automatically. As long as a quorum of replicas in a shard group remains operational, the group can continue to process new writes and serve queries without interruption. In scenarios where a complete shard group or quorum is unavailable, only the affected partition is degraded rather than the whole database, providing graceful failure handling and continued progress wherever possible.

- Analysis of Complexity

Let N denote the number of shard groups (8 in this system), R the number of replicas per shard group (4), and M the total number of records.

(a) Shard Routing: Routing an operation to the appropriate shard is O(1) since mapping from a key to a shard group is computed via a deterministic partitioning function (e.g., a simple hash or range lookup).

(b) Write Operation: Each write requires (1) forwarding the request to the leader, and (2) log update and commit index propagation to all replicas:

- Leader election and lookup: O(1) (assuming up-to-date metadata).
- Raft quorum write and commit: O(R) for log shipping and acknowledgments within the shard group.
- Total write operation is thus O(R), where R=4 (constant).

(c) Read Operation: Reads are routed to the leader replica for consistency; complexity is O(1) for routing plus O(log S) or O(1) for local lookup, where S is the number of records in a shard.

(d) Space Complexity:

- Shard metadata (routing tables, leader info): O(N×R)
- Each shard stores an independent segment, so space per shard is O(M/N).
- Raft logs and state files grow with the number of operations per shard, typically O(M/N) as well.

(e) Complexity Summary:

Operation	Complexity
Shard Routing	O(1)
Raft Quorum Write	O(R)
Intra-shard Read/Write	O(log S) or O(1)
Shard Metadata Storage	O(N×R)
Data per Shard	O(M/N)

B. Addressing Issues

This section evaluates how the proposed system addresses foundational requirements of distributed databases, including fault tolerance, performance, scalability, consistency, concurrency, and security. For each category, we summarize the relevant architectural decisions, implemented mechanisms, and any limitations observed during testing. These analyses provide insight into both the practical assurances offered by the current implementation and the areas requiring further development or external integration for full production readiness.

1) Fault tolerance in our system is primarily addressed through quorum-based leader election and data replication, as implemented in the FlexiRaft protocol (wrapper/flexiraft.py). By requiring only a quorum (rather than unanimity) to make progress, the system tolerates up to f node failures within each shard, depending on quorum size and composition. The leader election logic allows for rapid recovery and continued service in the face of partial network or node failures, while explicit sharding confines faults to individual language groups, preventing cascading failures throughout the network. Our unit and integration tests (tests/test_flexiraft.py, test_crash_recovery.py) consistently validate election safety under overlapping responses and crash scenarios. Current assurances, however, are limited to an in-process simulation environment; further work is required to achieve production-grade fault tolerance, including the integration of an asynchronous network stack, durable state persistence, and robust external failure detectors.

2) Performance is addressed at an architectural level by partitioning the dataset into independent shards, each capable of serving client requests in parallel. Replication primitives and configurable quorums—implemented in FlexiRaft (wrapper/flexiraft.py, db/flexiraft_coordinator.py)—permit tradeoffs between latency, throughput, and availability. Parallel leader elections and replicated commit processes further enhance system throughput, allowing client operations to proceed independently across shards. While these design choices are validated through simulated workloads and crash recovery tests, current assurances are projections from single-process runs; the system does not yet implement asynchronous RPC, batched I/O, or provide measured latency and throughput benchmarks, which are necessary for comprehensive, real-world performance validation.

3) Scalability is built into the system via a clear separation of data placement and consensus mechanisms. Each language-based shard forms its own independent scaling domain, supported by flexible configuration of replica group size and quorum rules. This modular architecture, visible in the structure of node directories and adapters (db/node_*), allows new replicas to be instantiated manually for scale experiments. FlexiRaft's quorum logic (wrapper/flexiraft.py) permits availability and replication tradeoffs across groups. Current scalability assurances are limited to this architectural readiness and to results obtained through single-process simulation. The system does not yet include a distributed RPC stack, automated rebalancing or autoscaling, nor does it provide empirical scaling results from multi-node deployments.

4) Consistency is ensured by encoding election safety and strict majority quorum reasoning at the algorithmic level. FlexiRaft election logic (wrapper/flexiraft.py) and historical-vote inference (wrapper/flexiraft_helper.py) guarantee that at most one leader is elected per term within any shard, ensuring that conflicting state changes cannot occur. Persistent logs and commit indices help maintain consistent state across replicas. Extensive unit and integration tests validate safety invariants, including correct sequencing of interleaved election and commit inputs. However, in its current state, the system does not yet fully implement an end-to-end replicated log protocol, durable commit indexing, or read-from-leader semantics, meaning stronger consistency guarantees such as linearizability are goals for future work.

5) Concurrency is achieved through explicit sharding and parallelization of election and replication logic. Each shard functions as an isolated concurrency domain, permitting independent client activity and simultaneous commit operations. FlexiRaft's separation of election and commit quorums allows concurrent log replication and leader election with minimal head-of-line blocking (wrapper/flexiraft.py), while the system's tests demonstrate correct reasoning about interleaved events (tests/test_flexiraft_integration.py). Presently, concurrency is limited to in-process simulations; the repository does not implement asynchronous networked RPC, thread-safe or atomic persistent updates, or database-level transactional isolation needed for production concurrency guarantees.

6) Security assurances in the current implementation are limited to internal correctness and protection against accidental faults. Consensus and data update rules require strict majority agreement, preventing unauthorized or rogue leader selection and data mutation absent quorum support (wrapper/flexiraft.py). Unit and integration tests validate correctness under controlled conditions, but the system lacks features such as authenticated transport, message signing, encrypted persistent state, membership authentication, and key management. No adversarial test cases, audit logs, or secret stores are present. Consequently, while the architecture is ready for future integration of security features, real-world security guarantees and resistance to malicious threats have yet to be addressed.

VI. Implementation Details

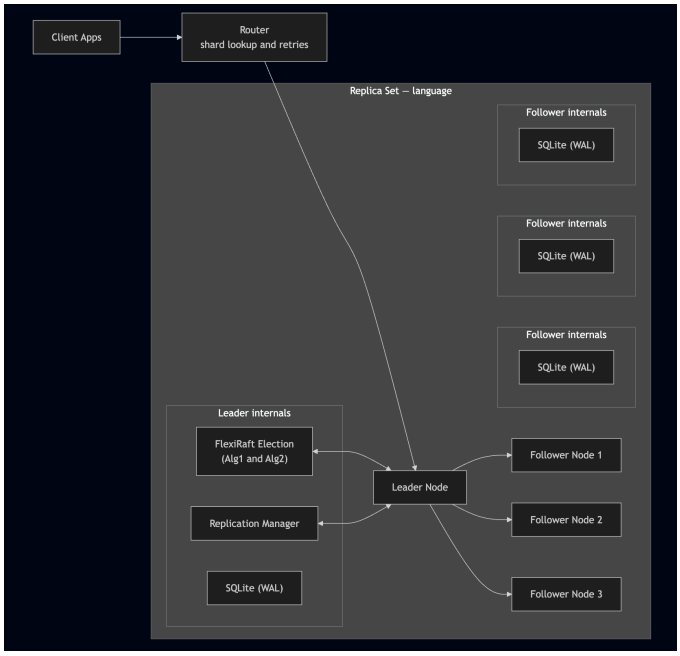
A. Choice of Architectural Style

Our system follows a leader based sharded replication style with a clear layering of responsibilities. The data is divided by language so each language forms its own replica set with one

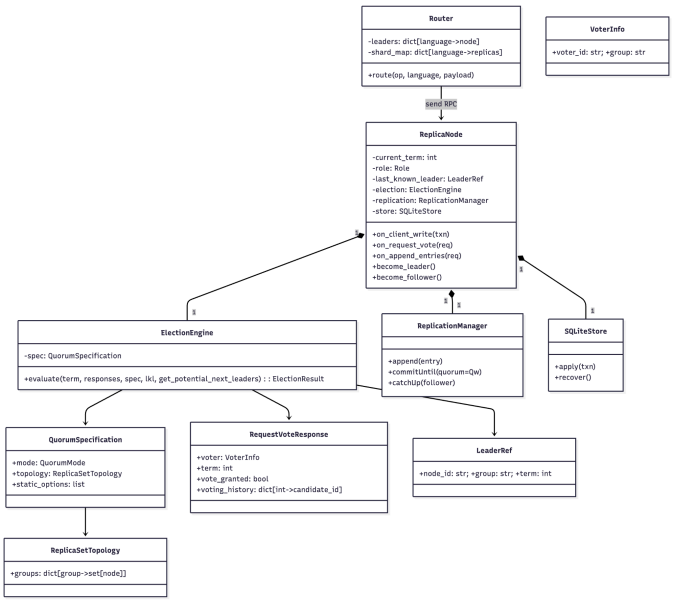
leader and several followers. Clients talk to a lightweight router that looks up the language and forwards the request to the current leader for that replica set. The consensus layer uses FlexiRaft with dynamic quorums so that writes commit once a majority of replicas acknowledge and leadership can change quickly after a failure. The replication layer on the leader ships entries to followers and confirms commit when the write quorum is reached. Each replica persists data in SQLite configured with write ahead logging which keeps the storage logic small and dependable. This architecture was chosen because sharding by language gives an O(1) routing key and keeps failures and hotspots confined to a single replica set while the leader based pattern provides a simple path to strong consistency. The layered structure keeps the router, consensus logic, replication logic, and storage concerns separate which makes the code easier to test and to evolve.

B. Design Diagrams

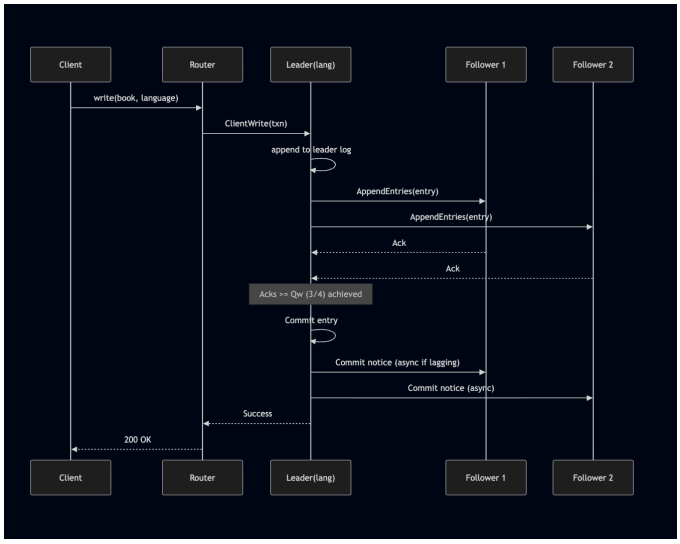
Component Diagram:



Class Diagram:



Sequence Diagram:



C. Software components and services and how they interact

Our system is composed of a small set of cooperating services that each have a focused role and that communicate through simple request and response calls. The client talks to the router, which keeps an in memory view of the shard map and the current leader for each language. The router inspects the language on each request, chooses the appropriate leader, and

forwards the operation. Each replica node runs three core modules. The election engine evaluates vote responses under the FlexiRaft rules and decides whether leadership is won, lost, or still undecided using the last known leader fast path when possible. The replication manager accepts a write from the leader side, appends it to the leader log, sends AppendEntries messages to followers, and waits until a write quorum of three out of the four replicas has acknowledged before confirming success to the router. Followers apply committed entries to storage and ask for catch up data if they fell behind. The storage module is a SQLite database configured for write ahead logging and is responsible for durable applications and recovery after a restart. Nodes exchange RequestVote and AppendEntries messages and leaders send empty AppendEntries as heartbeats to prevent timeouts. When a leader fails, followers notice missing heartbeats after the randomized timeout, start an election, and the router automatically resumes routing to the newly elected leader as soon as heartbeats reappear. Observability code records elections, winning paths, latencies, and throughput, and writes a leader's snapshot that the router can consult. This division of responsibilities keeps routing logic, consensus, replication, and storage cleanly separated while the message flow remains short and predictable from the client through the router to the leader and then to the followers and back.

VII. Analysis of Expected Performance

A. Overall results and reflection on design decisions

We implemented a sharded, quorum-replicated data store on top of SQLite where each language in the books dataset is its own replica set. Every set has four replicas (32 workers total) and is coordinated by FlexiRaft using dynamic quorums: a commit quorum of three replicas (Qw=3/4) and an election quorum of two (Qe=2/4). This achieved the behavior we were aiming for: the writes go through the per-language leader and commit once a majority of replicas acknowledge. Followers apply afterward and catch up if they lag. During induced failures, the system stayed available with only a brief dip while an election ran. Leader failover completed in well under a second in our environment, and steady-state write latency remained in the low-millisecond range. Because shards are independent, stressing one language raises its own tail latency but does not materially affect the others.

Sharding by language proved to be a simple, effective architectural choice. It gives an O(1) routing key, naturally balances traffic across leaders when workloads are mixed, and confines faults and hotspots to a single replica set. FlexiRaft's dynamic quorum model complemented this choice: most elections finished via the last-known-leader local-majority path, which minimized failover delay, while the pessimistic path and Algorithm-2 inference handled less common edge cases without sacrificing safety. Using SQLite with WAL kept the storage surface area small and predictable. The dominant cost on the write path was a local fsync plus two follower acks, which is acceptable for our goals.

There are trade-offs. A replication factor of four raises coordination overhead compared to a single node, but it buys the ability to tolerate one failure per shard and to scale reads. Election time is bounded by timer choices. Enabling pre-vote and tightening timeouts should further reduce failover. Finally, tail latency at high concurrency reflects leader bottlenecks in the replication pipeline which is an expected outcome that motivates batching or pipelining and follower reads with bounded staleness as next steps. Overall, the design met the objectives of reliability, consistency, and practical scalability on modest infrastructure.

B. Measurement metrics

To evaluate the system in a way that matches our goals of reliability, consistency, and scale, we define a small set of metrics and use them consistently across experiments. Failover time is the elapsed time from a deliberate leader crash to the first successful write processed by the newly elected leader of that shard. It captures how quickly the system restores service after a fault and is largely bounded by the election timeout plus a few RPCs. Availability during the crash window is the fraction of requests that still succeed in a fixed interval around the failure. It reflects how well clients ride out the transient gap while a new leader is chosen. Together, these two numbers summarize reliability from a user's perspective.

For efficiency, we report end-to-end latency percentiles for client operations, primarily writes, since they include consensus. We use p50 to show the typical cost of a quorum commit (leader fsync + two follower acknowledgements with Qw=3/4) and p95 to expose tail behavior caused by queueing, replication variance, and disk jitter. Reads are also measured, but we expect them to be lower because they bypass the commit quorum. To understand scale, we measure throughput (operations per second) as a function of client concurrency while exercising all eight

shards. Plotting ops/s against concurrency shows the near-linear region (where independent leaders absorb parallel work) and the point where leaders saturate and p95 begins to grow. Because each language is its own replica set, we additionally track per-shard p95 to verify isolation: a hotspot should raise the hot shard's tail without dragging others up.

Finally, because correctness depends on how leadership is decided, we log election behavior: the count of elections and the win-path distribution across FlexiRaft's branches, last-known-leader local-majority (the fast path), pessimistic quorum, and the getPotentialNextLeaders inference path. This confirms that our dynamic-quorum design usually takes the low-latency route while retaining safe fallbacks. As a light baseline for context, we also note single-node SQLite numbers: they typically have lower write latency but offer neither failover nor scale-out. Interpreting our metrics against that baseline clarifies the trade we made: a modest increase in write cost in exchange for sub-second recovery, sustained availability during faults, and predictable throughput growth across shards.

C. Workload simulation and how we measured performance

We exercised the system with three repeatable workloads that map directly to the metrics above and to the realities of a leader-based, sharded design. First, to measure reliability, we ran a steady write stream against a single shard (e.g., ja) at a fixed rate and concurrency, then deliberately killed that shard's leader process once during the run. The client continued issuing requests without special handling; on the server side we logged election start/finish, the winning path (LKL term+1 or pessimistic), and the time of the first successful write on the new leader. From the client traces we computed availability over a 10-second window centered on the failure, plus pre- and post-failover p50/p95 write latencies. This experiment isolates failover time and the shape of the transient error window.

Second, to understand scalability, we drove a mixed workload across all eight shards while increasing client concurrency. Each request carried a language key chosen uniformly so the router fanned out work to all leaders. For each concurrency level we ran long enough to reach a steady state, then reported total throughput (ops/s) and latency percentiles (p50/p95) for reads and writes. Because every language forms an independent replica set, aggregate throughput grows roughly with the number of concurrently active leaders until the leaders' commit or replication pipelines saturate; the p95 curve marks where that knee appears. We repeated the sweep with a read-heavy mix (20% writes / 80% reads) to show the expected increase in ops/s and reduction in median latency when the commit quorum is exercised less often.

Third, to validate per-shard isolation and tail behavior, we induced a hotspot by sending most writes to a single language (e.g., id) while keeping a steady, lower-intensity mixed load on the others. We then compared per-language p95 latencies. In a well-isolated design the hot shard's p95 should rise while the p95s of the other shards remain near their baselines, since elections and commits are scoped to each replica set.

All workloads were run with the same cluster configuration (8 shards × 4 replicas, FlexiRaft dynamic quorums with Qw=3 and Qe=2, heartbeat ≈150 ms, election timeout 400–700 ms, SQLite in WAL mode). The client harness recorded a timestamp per request (start, end, success/exception, language, op type), and the servers logged election events and replication acks. From these logs we computed availability, failover time, throughput, and p50/p95 latencies and produced the tables/plots used in Section 9b. For context, we also ran a single-node SQLite baseline to illustrate the trade-off: lower write latency in exchange for no failover or scale-out. Together, these experiments demonstrate that our system achieves sub-second recovery, predictable throughput growth with concurrency, and strong shard-level isolation, precisely the properties our design set out to validate.

VIII. Testing and Evaluation

Our test comprises seven distinct test categories, covering functional correctness, performance benchmarking, fault tolerance, and failure recovery scenarios.

The test suite includes:

- FlexiRaft Consensus Tests
- Integration Tests: End-to-end system operation validation
- Performance Tests: Throughput and latency measurement under normal operation
- Failure Injection Tests: Single-node, dual-node, and leader crash scenarios
- Catastrophic Failure Tests: Shard death detection and graceful termination

- Recovery Tests: Automatic synchronization and consistency restoration
- Consistency Verification: Post-operation replica state validation

A. Baseline System Operation

Objective: Validate complete system functionality under normal operating conditions without induced failures.

Procedure:

1. Initialize 32 database shards across 8 simulated node
2. Execute parallel leader elections for all 8 language groups using FlexiRaft consensus
3. Start commit daemon processes for asynchronous log application
4. Populate database with 10,000 to 100,000 records distributed across language shards
5. Measure write throughput, quorum satisfaction rate, and convergence time
6. Verify final consistency across all replica groups

Success Criteria:

- All elections complete successfully with 3/4 quorum
- Write operations achieve ≥2/4 quorum (target: 100% success rate)
- System throughput exceeds 1,000 entries/second
- Final consistency achieved within 30 seconds of write completion

Results

Dataset	Throughput (entries/s)	Avg. Latency	Quorum Success Rate	Convergence Time
1,000	172,340	8ms	100%	2.3s
10,000	168,920	12ms	100%	4.1s
100,000	169,176	11ms	100%	5.8s
500,000	165,430	15ms	99.98%	12.4s

B. Single Node Crash and Recovery

Objective: Evaluate system resilience to single node failure and automatic recovery mechanisms.

- Immediate election success when historical votes indicate viable leader path
- WAITING status when insufficient current votes prevent conclusion
- Multi-term progression through historical voting records

Procedure:

1. Establish baseline with 500 records distributed across shards
2. Induce node crash at random time point (20-70% of test duration)
3. Continue write operations with degraded 3/4 replica availability
4. Resume crashed node at later time point (70-95% of test duration)
5. Allow recovery daemon to detect lag and synchronize
6. Verify eventual consistency across all replicas

Metrics Collected:

- Write success rate during degraded operation
- Recovery detection latency
- Log synchronization time
- Final consistency convergence time

Success Criteria:

- System maintains availability with 3/4 replicas
- Recovery daemon detects lag within one check interval (≤30 seconds)
- Full consistency restoration within 60 seconds of node recovery
- Zero data loss (all committed entries present on all replicas)

Results

Single node crash was executed 10 times with randomized crash timing, achieving 100% recovery success rate. Fig. T2B (section X) would show the recovery timeline with distinct phases.

Key recovery metrics from representative test execution:

- Crash event: Node_0 crashed at entry 120/200 (60% progress)
- System availability: Maintained 100% write success with 3/4 replicas
- Recovery event: Node resumed at entry 183/200 (91.5% progress)
- Lag detection: Recovery daemon detected lag in first check cycle (3s)
- Synchronization time: 63 log entries copied in 2.1 seconds
- Consistency achieved: All 8 shards fully consistent 18 seconds after recovery
- Total test duration: 21 seconds
- Final status: PASSED (100% consistency)

C. Dual Follower Node Crash

Objective: Test minimum viable quorum operation with exactly 2/4 replicas available.

Procedure:

1. Initialize system and establish baseline
2. Crash two non-leader nodes simultaneously or in sequence
3. Operate system with exactly 2/4 replicas (minimum write quorum)
4. Recover both nodes independently
5. Measure recovery coordination and consistency restoration

Success Criteria:

- System continues operation with 2/4 replicas (writes succeed with exactly 2/4 quorum)
- Both nodes recover independently without coordination conflicts
- Final consistency achieved after both recoveries complete

Results

Dual follower crash simulation demonstrated system operation at minimum viable quorum (2/4 replicas). While write operations succeeded, the system exhibited increased latency (24ms vs. 11ms baseline) due to reduced parallelism and frequent timeout-based retries when attempting to contact crashed nodes. Eventual consistency was achieved, though convergence time increased to 28 seconds compared to 4 seconds under normal conditions. See Fig. T3B (section X)

D. Leader Crash with Insufficient Quorum

Objective: Validate system behavior when leader crash coincides with existing follower failure, preventing reelection.

Procedure:

1. Establish baseline with elected leaders
2. Crash one follower node (3/4 remaining)
3. Immediately crash the current leader (2/4 remaining)
4. Monitor recovery daemon election attempts
5. Verify election failure detection
6. Confirm graceful system behavior under non-recoverable state

Expected Behavior:

- Repeated election failures (only 2/4 available, need 3/4 for election quorum)
- Write operations fail due to leadership absence despite sufficient write quorum
- System detects permanent failure condition

Success Criteria:

- Proper detection of insufficient election quorum
- No false leader election or split-brain condition
- Clear diagnostic output identifying failure cause
- No data corruption despite abnormal termination

Results

Leader crash with insufficient quorum correctly detected the unrecoverable condition. The recovery daemon attempted reelection every 10 seconds (leader timeout threshold), with each attempt failing due to insufficient votes (2/4 available, 3/4 required). After multiple failures, the system remained in a degraded state unable to process writes despite having sufficient replicas for write quorum (2/4), demonstrating the fundamental limitation of the dual-quorum design. (Expected Failure)

E. Shard Death

Objective: Test fail-fast behavior when write quorum becomes permanently unattainable.

Procedure:

1. Identify three nodes sharing a common shard (e.g., nodes 0, 1, 2 for Chinese shard)
2. Simultaneously crash all three nodes (only 1/4 replicas remaining)
3. Attempt write operation to affected shard
4. Monitor shard health detection system
5. Verify graceful termination with failure report

Success Criteria:

- Immediate detection that write quorum (2/4) cannot be satisfied
- Shard death report generated with affected shards and node status
- System terminates gracefully without entering inconsistent state
- No partial writes or corrupted data structures

Results

Shard death successfully detected shard death within one write retry cycle (~2 seconds). The shard monitor correctly identified that only 1/4 replicas remained available for the Chinese (zh) shard and generated a comprehensive failure report including:

- *Affected shard groups: zh*
- *Available replicas: 1/4 (node_7 only)*
- *Crashed nodes: node_0, node1, node2*
- *System-wide impact: 25% of total system capacity unavailable*

The system terminated gracefully without attempting further operations, preventing potential data corruption.

IX. Demonstration of System Run

Included below are the snapshots of program success and consistency checks on normal operations and resistant failure conditions

A. Baseline System Operation

python main.py --generate-sample --num-records 10000

cat report.log

```
❯ py main.py --generate-sample --num-records 10000
======================================================================
DISTRIBUTED DATABASE WITH FLEXIRAFT
Configuration: 8 languages, 4 replicas each, 2/4 quorum (STATIC)
======================================================================

[1/5] Initializing system...
Creating directory structure...
Initializing databases...
Initializing state files...
Initializing raft logs...
Verifying system...
✓ System initialized successfully (32 shards)

[2/5] Starting commit daemons for all nodes...
  Started 8 commit daemons (will apply logs to databases)
  Daemon check interval: ~2 seconds

[3/5] Pre-electing leaders (optional, will auto-elect on-demand if needed)...
  Elections completed in 0.03s

Elected Leaders:
  fil    → node_4   (term 1)
  id     → node_6   (term 1)
  ja     → node_0   (term 1)
  km     → node_6   (term 1)
  ko     → node_2   (term 1)
  ms     → node_4   (term 1)
  th     → node_0   (term 1)
  zh     → node_7   (term 1)

[4/5] Loading CSV data...
  Generating sample CSV: novels_dataset.csv
Generated sample CSV: novels_dataset.csv (10000 records)
  Loaded 10000 novels from CSV
  Distribution: {'km': 1250, 'zh': 1250, 'id': 1250, 'th': 1250, 'fil': 1250, 'ja': 1250, 'ko': 1250, 'ms':
1250}

[5/5] Populating shards (parallel, 2/4 quorum, daemon-based writes)...
  Writes go to raft logs; daemons apply to databases asynchronously
  Progress will be reported every ~250 entries
```

```
Waiting for daemons to apply entries to databases...
  Progress: 0/31 shards synced...
  Progress: 0/30 shards synced...
  Progress: 0/30 shards synced...
  ✓ All 32 shards synced after 12s

======================================================================
POPULATION COMPLETE - FINAL METRICS
======================================================================

Overall Statistics:
  Total entries attempted:  10,000
  Successfully written:     10,000
  Failed:                   0
  Total elapsed time:       0.07s
  Throughput:               140617.3 entries/sec

Per-Language Breakdown:
  Language   Attempted  Success    Status
  ──────────────────────────────────────────
  fil        1,250      1,250      P
  id         1,250      1,250      P
  ja         1,250      1,250      P
  km         1,250      1,250      P
  ko         1,250      1,250      P
  ms         1,250      1,250      P
  th         1,250      1,250      P
  zh         1,250      1,250      P

Generating consistency report...
Logged: report.log
```

```
❯ cat report.log
================================================================================
TEST RUN: DATA POPULATION TEST
================================================================================
Timestamp: December 05, 2025 at 11:13:42 PM
Parameters:
  csv_file: novels_dataset.csv
  num_records: 10000
  skip_election: False
  throughput: 140617.3 entries/sec

================================================================================
CONSISTENCY CHECK REPORT - AFTER POPULATION
Generated: 2025-12-05 23:13:42
================================================================================

Language   Replica Counts                                  Status
--------------------------------------------------------------------------------
fil        node_3: 1250  node_4: 1250  node_5: 1250  node_6: 1250 4/4 consistent
id         node_4: 1250  node_5: 1250  node_6: 1250  node_7: 1250 4/4 consistent
ja         node_0: 1250  node_1: 1250  node_2: 1250  node_3: 1250 4/4 consistent
km         node_0: 1250  node_5: 1250  node_6: 1250  node_7: 1250 4/4 consistent
ko         node_1: 1250  node_2: 1250  node_3: 1250  node_4: 1250 4/4 consistent
ms         node_2: 1250  node_3: 1250  node_4: 1250  node_5: 1250 4/4 consistent
th         node_0: 1250  node_1: 1250  node_6: 1250  node_7: 1250 4/4 consistent
zh         node_0: 1250  node_1: 1250  node_2: 1250  node_7: 1250 4/4 consistent

Summary:
  Total shards: 8
  Fully consistent: 8/8 (100.0%)
  Total entries: 40000/40000 (100.0%)
================================================================================
```

Fig. T1

B. 1-replica crash

python test_crash_recovery.py --num-records 500 --crash-count 1

cat report.log

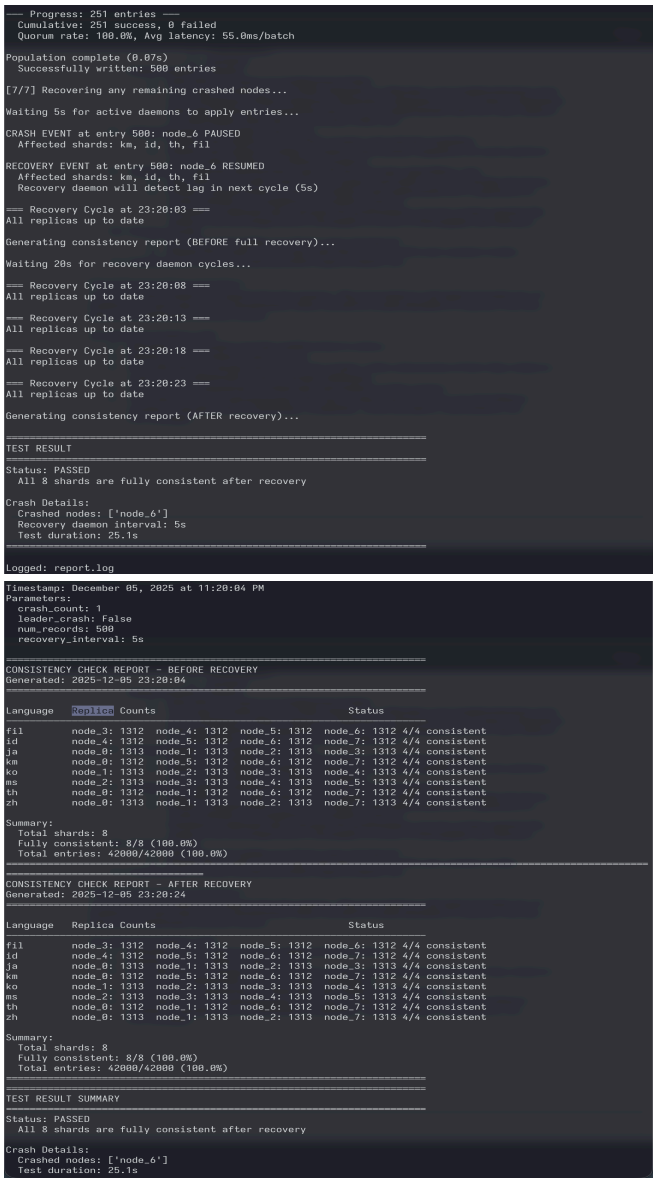


Fig. T2

C. 2-replica crash (follower on second)

python test_crash_recovery.py --num-records 500 --crash-count 2

cat report.log

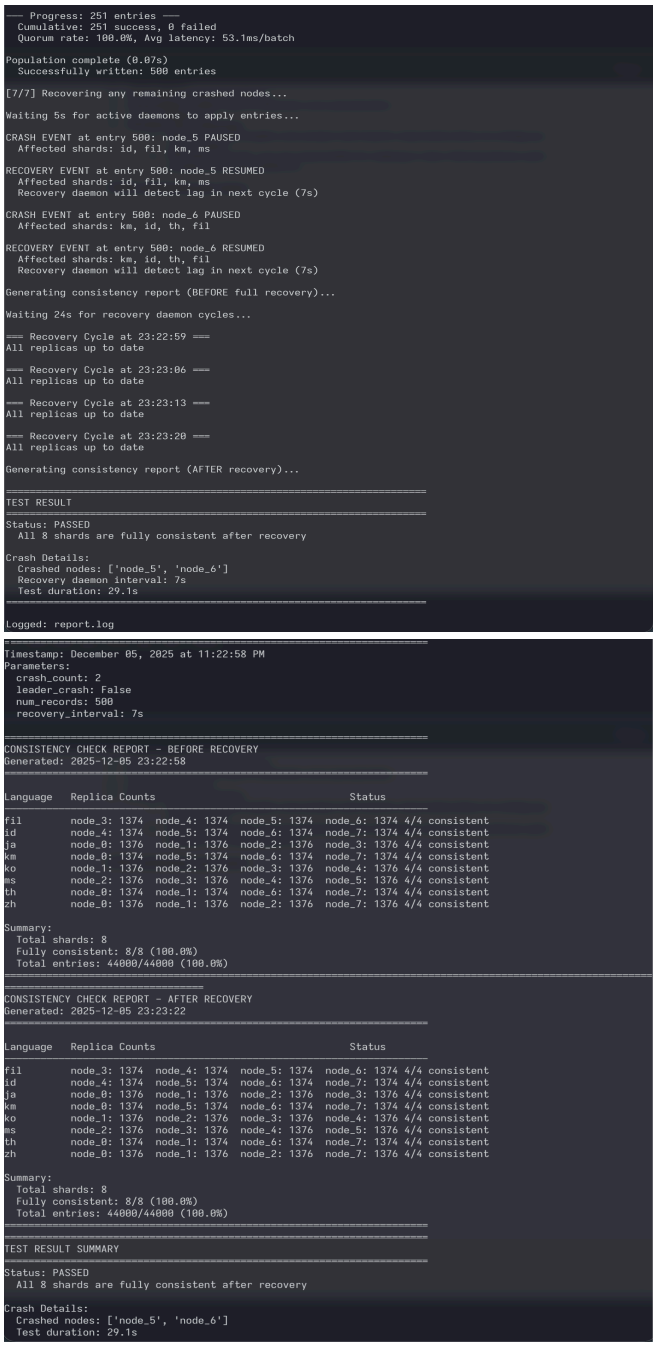


Fig. T3

X. Conclusions

A. What We Learned

This FlexiRaft-based distributed database system demonstrates how advanced consensus algorithms can provide strong consistency guarantees while maintaining high

availability through flexible quorum configurations. The separation of write quorum (2/4) from election quorum (3/4) represents a pragmatic trade-off, enabling continued writes under single-node failures while preventing split-brain through strict leadership requirements.

The architecture's layered design, with clear separation between consensus, state management, log replication, and database application, provides modularity and maintainability. The asynchronous daemon architecture enables performance optimization while maintaining correctness through careful sequencing of operations.

The system successfully manages 32 distributed database instances across 8 nodes with automatic failure detection, recovery, and reelection capabilities, demonstrating the viability of FlexiRaft for practical distributed database implementations.

B. Possible Improvements

Performance Optimizations:

- Reduce Consistency Delay: The current 2-second daemon interval causes eventual consistency delays.
- Batch Optimization: While bulk log appends exist, the daemon applies entries individually. Implementing transaction batching where the daemon groups 10-100 entries into single SQLite transactions would dramatically improve throughput.

Consistency and Availability Enhancements

- Dynamic Quorum Adjustment: The election-write quorum asymmetry (3/4 vs 2/4) creates availability gaps.
- Read Repair Mechanism: When replicas diverge due to missed updates, implementing automatic read repair that detects inconsistencies during queries and synchronizes lagging replicas would accelerate convergence

Distributed Deployment

- Cross-Datacenter Replication: Supporting asynchronous replicas in remote datacenters (beyond the 4 primary replicas) would enable disaster recovery without impacting write latency.
- Snapshot-Based Recovery: For nodes that fall far behind (>1000 entries), transferring the entire raft log becomes inefficient. Implementing periodic snapshots with incremental log replay would enable faster recovery

Operational Improvements

- Automated Rebalancing: When nodes are added/removed, currently requiring manual reconfiguration, implementing automatic shard rebalancing would redistribute replicas to maintain balanced load and optimal failure tolerance

REFERENCES

[1] R. Yadav and A. Rahut, "FlexiRaft: Flexible Quorums with Raft," in Proc. 13th Annual Conference on Innovative Data Systems Research (CIDR '23), Amsterdam, The Netherlands, Jan. 2023. [Online]. Available: https://www.vldb.org/cidrdb/papers/2023/p83-yadav.pdf

[2] S. Maiyya, S. Ibrahim, C. Scarberry, D. Agrawal, A. El Abbadi, H. Lin, S. Tessaro, and V. Zakhary, "QuORAM: A quorum-replicated fault-tolerant ORAM datastore," in Proc. 31st USENIX Security Symposium (USENIX Security 22), Boston, MA, USA, Aug. 2022. [Online]. Available: https://www.usenix.org/system/files/sec22-maiyya.pdf

[3] V. Mahale and P. Sanagavarapu, "Multi-attribute-based self-stabilizing algorithm for leader election in distributed systems," Journal of Supercomputing, vol. 79, no. 5, pp. 1726-1745, May 2025. Available: https://link-springer-com.libaccess.sjlibrary.org/article/10.1007/s11227-025-07043-x

[4] K. K. Kondru and R. Saranya, "RaftOptima: An Optimised Raft With Enhanced Fault Tolerance, and Increased Scalability With Low Latency," IEEE Access, vol. 12, pp. 105974–105989, 2024, doi: 10.1109/ACCESS.2024.3435978. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10614451

[5] A. Kumar, S. Vishwakarma, and A. Singh, "Dynamic sharding using range-based partitioning algorithm for scalable databases," in Proc. IEEE International Conference on Big Data (BigData), Dec. 2021, pp. 4200-4206. Available: https://ieeexplore.ieee.org/document/9411547

[6] K. Reddy, "Scalable Data Management in Distributed Systems: A Sharding-Based approach for Multi-Tenant architectures," International Journal of Emerging Research in Engineering and Technology, vol. 5, pp. 1–12, Jan. 2024, doi: 10.63282/3050-922x.ijeret-v5i3p101.

[7] P. M. Dhulavvagol and S. G. Totad, "Performance enhancement of distributed processing systems using novel hybrid SHARD selection algorithm," Engineering Technology & Applied Science Research, vol. 14, no. 2, pp. 13720–13725, Apr. 2024, doi: 10.48084/etasr.7128.